NOI 2026 Final Contest

14 March 2026

Tasks	Time Limit	Memory Limit
Monkey	1 s	1024 MB
Famished Cats	1 s	1024 MB
3 Raptors	2 s	1024 MB
Gemstones	2 s	1024 MB
Lemon	5 s	2048 MB

Have Fun!

Task 1: `Monkeys`

Monkeyland is an infinite number line with n monkeys, numbered from 1 to n. The i-th monkey is initially at position $p[i]$ on the number line. It is possible that multiple monkeys share the same initial position.

Pan can make every monkey move with his enchanting spell! How each monkey moves is determined by a string d of length n where each character is either `L` or `R`. Let the i-th character of d be $d[i]$.

Once a spell is cast, the i-th monkey moves as follows:

- If $d[i] =$ `L`, it moves to the left by **one position**.
- If $d[i] =$ `R`, it moves to the right by **one position**.

Each day, Pan will cast his spell exactly once. If two monkeys are at the same position on any day (even initially), they become **friends**. If Pan were to cast his spell for k days, determine the number of **pairs of monkeys** that will become friends.

Input format

Your program must read from standard input.

The first line of input contains two space-separated integers n and k.

The second line of input contains n space-separated integers $p[1], p[2], \ldots, p[n]$

The third line of input contains a string d of n characters $d[1], d[2], \ldots, d[n]$.

Output format

Your program must print to standard output.

Output one integer, the number of pairs of monkeys that will become friends.

The output should contain only a single integer. Do not print any additional text such as `Enter a number` or `The answer is`.

Subtasks

For all test cases, the input will satisfy the following bounds:

- $1 \leq n \leq 200\,000$
- $1 \leq k \leq 10^9$
- $1 \leq p[i] \leq 10^9$ for all $1 \leq i \leq n$
- $d[i]$ is either `L` or `R` for all $1 \leq i \leq n$

Your program will be tested on input instances that satisfy the following restrictions:

Subtask	Score	Additional constraints
0	0	Sample test cases
1	6	$n = 2$
2	13	$d[1] = d[2] = \cdots = d[n]$
3	10	$n, k \leq 200$
4	22	$n, k \leq 3000$
5	18	$n \leq 3000$
6	31	No additional constraints

Sample Test Case 1

This test case is valid for subtasks 1, 3, 4, 5, and 6.

Input	Output
2 1 1 3 RL	1

Sample Test Case 1 Explanation

There are $n = 2$ monkeys and Pan casts the spell for $k = 1$ day only.

On the first day, monkey 1 moves right from position 1 to position 2 while monkey 2 moves left from position 3 to position 2. Since they end up at the same position on the first day, they become friends. Hence, there is exactly 1 pair of monkeys that become friends.

Sample Test Case 2

This test case is valid for subtasks 2, 3, 4, 5, and 6.

Input	Output
5 67 1 2 3 4 5 RRRRR	0

Sample Test Case 2 Explanation

There are $n = 5$ monkeys and Pan casts the spell for $k = 67$ consecutive days.

Since all monkeys have distinct initial positions and each monkey moves one position to the right each day when a spell is cast, no two monkeys will be at the same position on any day. Hence, no pairs of monkeys can become friends.

Sample Test Case 3

This test case is valid for subtasks 3, 4, 5, and 6.

Input	Output
6 7 1 1 8 16 18 22 RRLRLL	3

Sample Test Case 4

This test case is valid for subtasks 3, 4, 5, and 6.

Input	Output
10 30 9 46 27 8 12 100 56 96 6 7 LRLRRLRRLR	5

Sample Test Case 5

This test case is valid for subtasks 3, 4, 5, and 6.

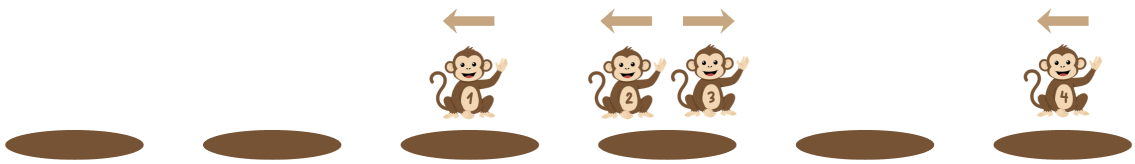
Input	Output
4 2 3 4 4 6 LLRL	2

Sample Test Case 5 Explanation

There are $n = 4$ monkeys and Pan casts his spell for $k = 2$ consecutive days.

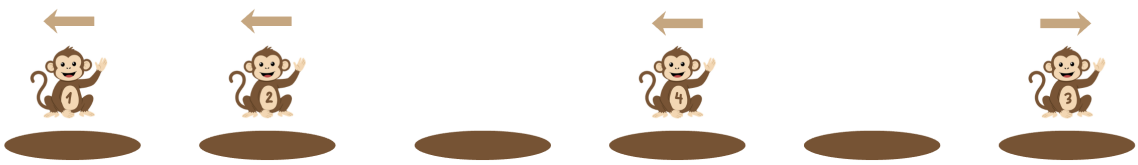
Each figure below depicts Monkeyland as a number line showing positions from 1 to 6 **only**. The arrow above each monkey indicates the direction in which it will move after a spell is cast.

On the 0-th day, the initial positions of all monkeys are shown in the figure below. Monkeys 2 and 3 which are already at position 4 become friends.



After the spell has been cast on the 1-st day, the positions of all monkeys are shown in the figure below. Monkeys 3 and 4 meet at position 5 and become friends.

After the spell has been cast on the 2-nd day, the positions of all monkeys are shown in the figure below. No two monkeys meet on this day.



Therefore, there are a total of 2 pairs of monkeys that are friends: Monkeys 2 and 3, as well as monkeys 3 and 4.

Task 2: `Famished Cats`

After successfully preventing the extinction of cats during the annual National Cat Day (NCD) celebration, Ket the cat has now received hunger complaints from the kingdom of cannibalistic cats. Hence, Ket has been tasked with delivering food to prevent them from resorting to cannibalism again.

This cat kingdom can be modelled as a very long road that runs from west to east. There is a food bank at the west end of the road. There are n cat houses to the east of the food bank, numbered from 1 to n. It is guaranteed that n is **even**. The first house is $d[1]$ kilometres east of the food bank. For $i \geq 2$, the i-th house is $d[i]$ kilometres east of the $(i-1)$-th house.

Ket will drive a delivery truck to deliver food to the houses. The truck starts from the food bank and initially has x units of fuel. 1 unit of fuel allows Ket to drive his truck 1 kilometre along the road.

The i-th house has $f[i]$ units of fuel for the truck to use. The truck has infinite fuel storage and stops only when it runs out of fuel. The truck does not need to return to the food bank after leaving.

In addition, Ket has a magic wand. In one wave of his wand, he can **swap** the amount of fuel stored in the i-th and $(n-i+1)$-th cats' houses. This swap can only take place if the fuels in both houses have not been used yet. Help Ket find the index of the farthest house D he can reach, using any number of swaps. Also, help Ket find the minimum number of swaps S required to reach house D.

You may refer to Sample Test Case 1 for a more detailed explanation.

Input format

Your program must read from standard input.

The first line of input contains two space-separated integers n and x.

The second line of input contains n space-separated integers $d[1], d[2], \ldots, d[n]$.

The third line of input contains n space-separated integers $f[1], f[2], \ldots, f[n]$.

Output format

Your program must print to standard output.

Output **two** space-separated integers on a single line. The first integer should be D, the index of the furthest house Ket can reach, followed by S, the minimum number of swaps required to reach house D.

Subtasks

For all test cases, the input will satisfy the following bounds:

- $2 \leq n \leq 500\,000$
- n is even.
- $1 \leq d[i] \leq 10^9$ for all $1 \leq i \leq n$
- $1 \leq f[i] \leq 10^9$ for all $1 \leq i \leq n$
- $d[1] \leq x \leq 10^9$

Your program will be tested on input instances that satisfy the following restrictions:

Subtask	Score	Additional constraints
0	0	Sample test cases
1	7	$\lvert f[i] - f[n-i+1] \rvert = k$ for some constant k, for all $1 \leq i \leq n$
2	12	$n \leq 40$
3	14	f is non-decreasing ($f[i] \leq f[i+1]$ for all $1 \leq i \leq n-1$)
4	19	$D \leq \frac{n}{2}$
5	21	$n \leq 5000$
6	27	No additional constraints

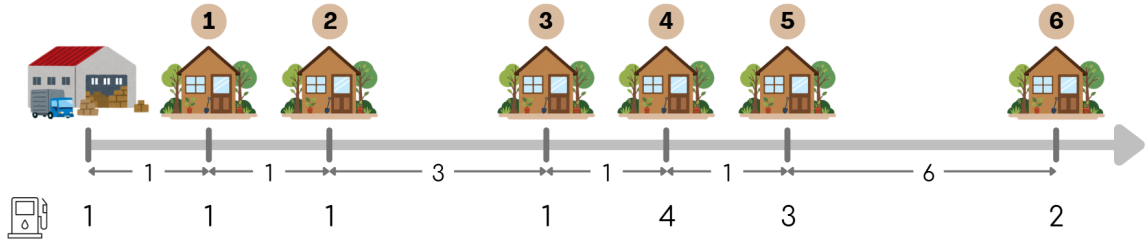
Note: The absolute value of a number x, denoted $|x|$, is the non-negative number equal to the distance between 0 and x on a number line. For example, $|5| = 5$ and $|-5| = 5$.

Sample Test Case 1

This test case is valid for subtasks 2, 5, and 6.

Input	Output
6 1 1 1 3 1 1 6 1 1 1 4 3 2	5 1

Sample Test Case 1 Explanation



There is a food bank with $n = 6$ houses to its east. Ket's truck starts with $x = 1$ unit of fuel and moves to house 1. He then refuels at house 1 and drives to house 2. At this point, it is optimal for Ket to use his magic wand to swap the fuel amounts in houses 2 and 5, allowing him to refill 3 units of fuel and reach house 3. Subsequently, he can refill 1 unit of fuel before travelling to the next two houses, refilling 4 and 1 units (due to the earlier swap) of fuel in houses 4 and 5 respectively.

He will then have 4 units of fuel left, leaving him unable to travel to house 6 as it requires 6 units of fuel. Since Ket runs out of fuel before reaching house 6, he can only travel to house 5. Furthermore, he had to make one swap with his magic wand. Therefore, $D = 5$ and $S = 1$.

Sample Test Case 2

This test case is valid for subtasks 1, 2, 5, and 6.

Input	Output
6 5 3 8 3 1 4 1 2 7 1 6 2 7	6 1

Sample Test Case 3

This test case is valid for subtasks 2, 3, 4, 5, and 6.

Input	Output
6 2 2 24 25 40 5 11 4 12 14 16 20 30	3 2

Sample Test Case 4

This test case is valid for subtasks 2, 5, and 6.

Input	Output
6 10 3 6 3 7 8 6 4 3 1 7 1 6	5 1

Task 3: `3 Raptors`

WhiteRaptor has finally found his own kind in TheRaptorLand! Unlike the plain WhiteRaptor, TheRaptorLand has an assortment of coloured raptors: the PinkRaptor, BlueRaptor and GreenRaptor.

WhiteRaptor lined up all n raptors in TheRaptorLand in a line, numbered from 1 to n from left to right. The i-th raptor from the left has colour $c[i]$. He wants to choose some raptors to keep him company in his basement forever. However, he is only able to do so by removing zero or more raptors from the left and right ends of the line, and keeping all raptors that remain.

To ensure that no raptors that remained will be ostracised, he wants the difference between the frequency of the most common raptor colour and the frequency of the least common raptor colour to not exceed k. Note that if no raptors of a certain colour remained, the frequency of the least common raptor colour will be 0.

Help WhiteRaptor find the maximum number of raptors that he can keep!

Input format

Your program must read from standard input.

The first line of input contains two integers n and k.

The second line of input contains n space-separated integers $c[1], c[2], \ldots, c[n]$.

Output format

Your program must print to standard output.

Output one integer, the the maximum number of raptors that he can keep.

Subtasks

For all test cases, the input will satisfy the following bounds:

- $1 \leq n \leq 200\,000$
- $0 \leq k \leq 200\,000$
- $1 \leq c[i] \leq 3$ for all $1 \leq i \leq n$

Your program will be tested on input instances that satisfy the following restrictions:

Subtask	Score	Additional Constraints
0	0	Sample test cases
1	5	$n \leq 500$
2	9	$n \leq 2000$
3	11	$c[i] \leq 2$
4	15	$k = 0$
5	16	There exists a $1 \leq j \leq n$ such that $c[i] \neq 3$ for all $i \leq j$, and $c[i] = 3$ for all $i > j$
6	20	In any contiguous sequence of 3 or more raptors, colour 3 is the least common.
7	24	No additional constraints

Sample Test Case 1

This test case is valid for subtasks 1, 2, 6, and 7.

Input	Output
11 2 2 2 1 2 1 3 2 1 2 1 1	7

Sample Test Case 1 Explanation

From raptor 3 to raptor 9, the number of raptors with $c[i] = 1, 2,$ and 3 are 3, 3, and 1 respectively. Since the difference between the maximum and minimum frequencies does not exceed $k = 2$, this set of raptors satisfies WhiteRaptor's criterion.

An example of an invalid set of raptors is from raptor 3 to raptor 10, as the addition of another raptor with $c[i] = 1$ causes the frequency of the most common colour to become 4. The resulting difference between the maximum and minimum frequencies is 3, which exceeds $k = 2$.

It can be shown that 7 is the largest number of raptors WhiteRaptor can keep that still satisfies his criterion.

Sample Test Case 2

This test case is valid for subtasks 1, 2, 5, and 7.

Input	Output
6 2 2 1 3 3 3 3	5

Sample Test Case 2 Explanation

WhiteRaptor should choose the raptors from raptor 1 to raptor 5.

Sample Test Case 3

This test case is valid for all subtasks.

Input	Output
7 0 1 2 1 2 1 2 1	0

Sample Test Case 3 Explanation

Since any contiguous sequence of raptors will not contain a raptor with $c[i] = 3$, the frequency of the least common colour will always be 0. This means that WhiteRaptor cannot select any non-empty sequence of raptors. Hence, the output is 0.

Note that this testcase satisfies Subtask 5 because we can assign $j = n$ (no colour-3 raptors will appear).

Task 4: `Gemstones`

You are playing a puzzle game featuring n gemstones in a row, numbered from 1 to n from left to right. The i-th gemstone has colour $c[i]$.

At any point, you may select two adjacent gemstones of the same colour and delete them. Then, the gemstones on either side slide together to close the gap, possibly creating new adjacent pairs.

You will be given q independent scenarios. In the j-th scenario, you will only consider gemstones starting from gemstone $l[j]$ and ending at gemstone $r[j]$. Assuming you perform an optimal sequence of deletions, what is the minimum number of gemstones that can be left behind?

Input Format

Your program must read from standard input.

The first line of input contains two space-separated integers n and q.

The second line of input contains n space-separated integers $c[1], c[2], \ldots, c[n]$.

The next q lines of input each contain two space-separated integers. The i-th of these lines contains $l[i]$ and $r[i]$.

Output format

Your program must print to standard output.

The output should contain q lines. The i-th of these lines should contain one integer, the answer to the i-th scenario.

Subtasks

For all test cases, the input will satisfy the following bounds:

- $1 \leq n \leq 10^6$
- $1 \leq q \leq 500\,000$
- $1 \leq c[i] \leq 10^9$ for all $1 \leq i \leq n$
- $1 \leq l[j] \leq r[j] \leq n$ for all $1 \leq j \leq q$

Your program will be tested on input instances that satisfy the following restrictions:

Subtask	Score	Additional Constraints
0	0	Sample test cases
1	2	$c[1] = c[2] = \cdots = c[n]$
2	5	Gemstones of the same colour form a contiguous subarray (If $c[i] = c[j]$ and $i < j$, then $c[i] = c[i+1] = \cdots = c[j]$)
3	9	$n, q \leq 2000$
4	4	$l[j] = 1$ for all $1 \leq j \leq q$
5	8	There are exactly two gemstones of each colour
6	16	$c[i] \leq 2$ for all $1 \leq i \leq n$
7	18	$n, q \leq 100\,000$
8	15	$n, q \leq 300\,000$
9	23	No additional constraints

Sample Test Case 1

This test case is valid for subtasks 3, 7, 8, and 9.

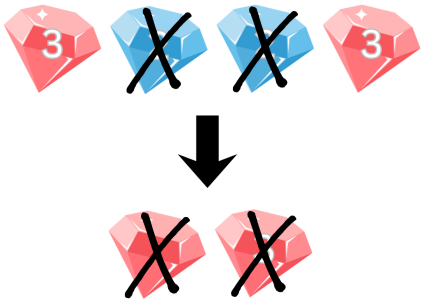
Input	Output
8 4 3 3 3 2 2 3 4 7 1 3 3 6 1 7 5 8	1 0 1 4

Sample Test Case 1 Explanation

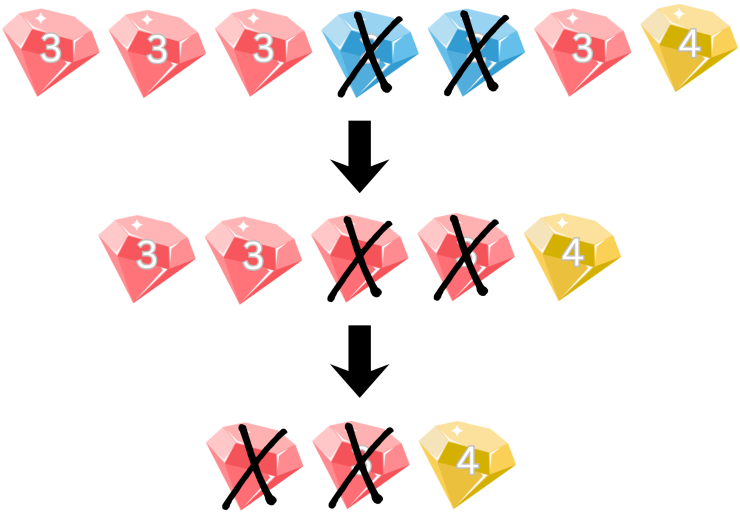
The $n = 8$ gemstones are shown in the diagram below.

In the first scenario, only the first three gemstones should be considered. Deleting any two adjacent gemstones will leave exactly one behind, after which it is impossible to delete any more gemstones. Hence, the answer is 1.

In the second scenario, gemstones can be deleted in the following manner, leaving none behind:



In the third scenario, gemstones can be deleted in the following manner, leaving one behind:



In the fourth scenario, no gemstones can be deleted. Hence, the answer is 4.

Sample Test Case 2

This test case is valid for subtasks 3, 6, 7, 8, and 9.

Input	Output
6 3 2 1 1 2 2 1 1 6 1 4 3 6	2 0 0

Task 5: `Lemon`

This is a communication task. Note that only C++ is allowed for this task.

Takina and Chisato are at a fruit convention. After a day of taking photos with their favourite fruit cosplayers, they came across a game booth.

The rules of the game are as follows:

- There are n fruits, each with a distinct label from 1 to n.
- Exactly one of the fruits is a lemon, but neither Takina nor Chisato know which one it is yet.
- Takina will be given all n fruits one by one. Her goal is to communicate the lemon's label to Chisato (who is blindfolded during this process).

Before receiving any fruits, Takina is given an array p, which represents the order in which the fruit labels will appear. $p[1]$ will be the label of the first fruit, $p[2]$ will be the label of the second fruit, and so on. Then, Takina will write down a binary string b containing only `0`s and `1`s. The string must not be longer than 5000 characters, but it can be empty. Let x denote the length of b.

Afterwards, the fruits are given to Takina one by one in the aforementioned order. Upon receiving each fruit, Takina is informed whether it is the lemon.

- If the fruit is **not** the lemon, Takina may choose whether or not to eat it. This decision must be made before receiving the next fruit and cannot be changed.
- If the fruit **is** the lemon, Takina must not eat it.

Let y denote the total number of fruits eaten by Takina.

Finally, Chisato receives the string b and the **sorted list of labels** of fruits that were uneaten. Using this information, Chisato must determine the label of the fruit which is the lemon.

Takina and Chisato have decided to play this game t times. Design a strategy for them to correctly identify the lemon while minimising both x and y.

Implementation Details

This is a communication task. Do not read from standard input or write to standard output.

You need to implement **three** procedures.

For Takina, you should implement:

```
std::string init(int subtask, int n, std::vector<int> p)
```

- `subtask`: the index of the subtask to which the test case belongs.
- n: the number of fruits.
- p: an array of length $n + 1$ where:
  - $p[0] = 0$, and
  - for each $1 \leq i \leq n$, $p[i]$ is the label of the i-th fruit that will be given to Takina.
- This procedure is called t times per test case, once at the start of each game.

This procedure should return string b, with length between 0 and 5000 inclusive, consisting of only `0`s and `1`s. If a string of invalid length or format is returned, you will receive a `Wrong Answer` verdict.

```
bool receive_fruit(int id, bool is_lemon)
```

- `id`: the label of the fruit given to Takina.
- `is_lemon`: `true` if the fruit given is the lemon, `false` otherwise.
- This procedure is called n times for each of the t games, once for each fruit.

This procedure should return `true` if the fruit should be eaten, and `false` otherwise. If `true` is returned when `is_lemon` is `true`, you will receive a `Wrong Answer` verdict.

For Chisato, you should implement:

```
int answer(int subtask, int n, std::string b,
        std::vector<int> uneaten)
```

- `subtask`: the index of the subtask to which the test case belongs.
- n: the number of fruits.
- b: the string returned by `init`.
- `uneaten`: the sorted vector of length $n - y + 1$ of labels of fruits not eaten by Takina, where:
  - `uneaten[0]` $= 0$, and
  - `uneaten[i]` is the i-th smallest label among uneaten fruits.
- This procedure is called t times per test case, once at the end of each game.

This procedure should return an integer, the label of the lemon. If the return value does not match the correct label, you will receive a `Wrong Answer` verdict.

In the actual grading, a program that calls the above procedures is run **twice**:

1. In the first run, the following steps are performed t times:
   - `init` is called once.
   - `receive_fruit` is called n times, following the order of fruits given to Takina.
   - Your program can store and retain information across successive calls.
2. In the second run, **the order of the games may be different from the first run**. For each of the t games:
   - `answer` is called once. Other than the parameters passed into `answer`, your program may not access information from the first run.

Since the procedure can be called more than once, you should pay attention to the effect of the remaining data from the previous call on the current call.

Subtasks

For all test cases, the input will satisfy the following bounds:

- $1 \leq t \leq 10\,000$
- $n = 500$
- $1 \leq p[i] \leq n$ for all $1 \leq i \leq n$
- There is exactly one lemon.

For each subtask, your program will be scored differently based on the length of the string Takina writes, x, and the number of fruits she eats, y. Your score for each test case is calculated using the maximum value of x in all t games and the maximum value of y in all t games.

Subtask	Score
1	If $y > 2$, your score is 0. Otherwise, your score is $10 \times \min\left(\frac{288}{x}, 1\right)$.
2	If $y > 9$, your score is 0. Otherwise, your score is $30 \times \min\left(\frac{30}{x}, 1\right)$.
3	Your score is $60 \times \min\left(\frac{20}{x+y}, 1\right)$.

Sample Interaction

Consider a single game ($t = 1$) with $n = 4$. Note that this does not satisfy the input constraints and is only used here for demonstration purposes.

Suppose that Takina is given the order of fruits $p = [0, 3, 1, 4, 2]$.

This means that the fruits will be presented in the following order of labels: $3, 1, 4, 2$.

Assume that the fruit with label 4 is the lemon.

First, `init` is called. Takina receives the parameters `subtask`, n, and p, and returns string b.

Then, `receive_fruit` is called once for each fruit in the given order. Each time, Takina is informed whether the fruit is the lemon and decides whether to eat it.

Finally, Chisato receives the string b and the sorted set of labels of fruits that were not eaten, and the procedure `answer` is called.

One possible sequence of calls and return values is shown below.

Step	Procedure Call	Parameters	Return Value
1	`init`	$(\texttt{subtask}, 4, [0, 3, 1, 4, 2])$	`"101"`
2	`receive_fruit`	$(3,$ `false`$)$	`true`
3	`receive_fruit`	$(1,$ `false`$)$	`false`
4	`receive_fruit`	$(4,$ `true`$)$	`false`
5	`receive_fruit`	$(2,$ `false`$)$	`true`
6	`answer`	$(\texttt{subtask}, 4, \texttt{"101"}, [0, 1, 4])$	4

In this example, Takina eats the fruits with labels 3 and 2, so the uneaten fruits are $\{1, 4\}$. The vector `uneaten` passed to `answer` is therefore $[0, 1, 4]$.

Using b and `uneaten`, the procedure `answer` returns 4, which is the label of the lemon. This strategy was able to correctly identify the lemon with $x = 3$ and $y = 2$.

Testing

You may download the sample grader (`grader.cpp`), the header file (`lemon.h`) and a solution template (`lemon.cpp`) under the Attachments. Two input files are provided for your testing, `sample1.txt` and `sample2.txt`. You may use the scripts `compile.sh` to compile and `run.sh` to run your solution for testing.

CMS user tests are not supported for this problem.

Sample Grader

A sample grader is provided to help test your implementation locally. Unlike the official grader, the sample grader runs your program only **once** and does not alter the order of tests for Takina and Chisato.

Input Format

The first line of input contains one integer `subtask`.

The second line of input contains one integer t.

The next t lines of input each describe one game. Each game is described by:

- a line containing two space-separated integers n and l, representing the number of fruits and the label of the lemon, and
- a line containing n space-separated integers $p[1], p[2], \ldots, p[n]$.

```
subtask
t
n_1 l_1
p_1[1] p_1[2] ... p_1[n_1]
n_2 l_2
p_2[1] p_2[2] ... p_2[n_2]
...
n_t l_t
p_t[1] p_t[2] ... p_t[n_t]
```

Output Format

The sample grader outputs a single real number representing the score fraction calculated from the values of x and y over all games.

Note

Additional diagnostic messages may be printed to standard error. The sample grader does **not** simulate the behaviour of the official grader.